Projekat iz industrijske robotike

1) Uvod

U ovom projektu realizovan je kompletan sistem upravljanja robotskim manipulatorom Franka Emika Panda korišćenjem:

- MuJoCo biblioteke za simulaciju dinamike,
- Pinocchio biblioteke za kinematiku i dinamiku robota.

Cilj projekta bio je da robot prati željenu poziciju i orijentaciju TCP-a, da glatko prelazi između dve poze u prostoru (dva targeta), da koristi realističan dinamički model i da bude upravljan momentima u zglobovima. Posebno važan deo projekta jeste činjenica da se trajektorija generiše u operacionom prostoru dok se upravljanje vrši u prostoru zglobova. To znači da se željeno kretanje prvo definiše kao željena pozicija i orijentacija TCP-a, a zatim se preko inverzne kinematike prvo pretvara u željene uglove u zglobovima i na kraju pomoću inverse dynamics kontrolera pretvara u momente koji se zadaju motorima.

2) Operacioni prostor i prostor zglobova

Kod robota postoje dva osnovna prostora: prostor zglobova (joint space) i operacioni prostor (task space).

Kod prostora zglobova, robot se opisuje uglovima u zglobovima.

$$q = [q1 \quad q2 \quad q3 \dots q_n]^T$$

U slučaju Panda robota imamo 7 zglobova tj. n = 7.

Operacioni prostor opisuje poziciju i orijentaciju TCP-a. U slučaju našeg projekta početni i krajnji target su predstavljeni pomoću matrice homogene transformacije.

$$T = \begin{bmatrix} R_{3x3} & p_{3x1} \\ 0\,0\,0 & 1 \end{bmatrix}$$

3) Sinteza trajektorije

Kao što je već rečeno ulazni podaci za sintezu trajektorije su početna i željena pozicija i orijentacija TCP-a koji su u vidu matrice homogene transformacije T_{start} i T_{goal} definisani od strane korisnika.

Sinteza pozicije

Iz datih matrica su izdvojeni vektori translacije kao x_{start} i x_{goal} koji predstavljaju početnu i krajnju poziciju TCP-a. Pomoću datih vektora se formira relativni vektor kao :

$$d = x_{goal} - x_{start}$$

Ovaj vektor ***d*** pokazuje smer kretanja i udaljenost između targeta. Moguće je izračunati ukupnu dužinu putanje kao normu datog vektora.

$$L_{total} = ||d||$$

Takođe je moguće odrediti i jedinični vektor pravca koji predstavlja samo smer.

$$\hat{e} = \frac{d}{||d||}$$

Ovo je veoma bitno jer sada možemo smer da držimo konstantnim tj. nepromenjenim, a da interpoliramo samo skalar tj. dužinu puta. Interpolacija se vrši polinomom petog stepena.

Željenu poziciju računamo po sledećoj formuli:

$$x_d = x_{start} + \hat{e} * L(t)$$

gde je x_{start} vektori translacije koji predstavljaju početnu poziciju TCP-a odakle smo krenuli, član $\hat{e}$ predstavlja fiksan ort vektor, dok je $L(t)$ interpoliran skalar dužine puta u svakom vremenskom trenutku trajektorije.

Sinteza orijentacije

Orijentacija je komplikovanija od pozicije jer ne možemo jednostavno oduzimati matrice rotacije pošto nisu linearne veličine. Zbog toga koristimo angle-axis reprezentaciju gde svaka matrica rotacije može da se predstavi kao osa i ugao rotacije. Iz početnih matrica

homogene transformacije T_{start} i T_{goal} definisani od strane korisnika, izdvojene su matrice rotacije R_{start} i R_{goal}. Ove matrice predstavljaju početnu i krajnju orijentaciju TCP-a. Pomoću datih matrica se formira relativna matrica koja govori koliko treba TCP da se rotira od početne do krajnje orijentacije.

$$R_{rel} = R_{start}^{T} * R_{goal}$$

Iz date matrice možemo da odredimo rotacioni vektor pomoću logaritamske mape. U *pinocchio* biblioteci se to radi pomoću funkcija *pin.log3*.

$$r = pin.log3(R_{rel})$$

Norma datog vektora ***r*** predstavlja ugao rotacije dok jedinični vektor predstavlja jediničnu osu rotacije.

$$\theta = ||r||$$

$$a = \frac{r}{||r||}$$

Isto kao i kod sinteze pozicije, ovo je veoma bitno jer sada možemo smer da držimo konstantnim tj. nepromenjenim, a da interpoliramo samo skalar tj. ugao rotacije. Interpolacija se takođe vrši polinomom petog stepena.

Željenu rotaciju računamo po sledećoj formuli:

$$R_d = R_{start} @ pin.exp3(a * \theta(t))$$

gde je R_{start} matrica rotacije koja predstavlja početnu orijentaciju TCP-a odakle smo krenuli, član a predstavlja fiksan ort vektor, dok je $\theta(t)$ interpoliran skalar ugla rotacije u svakom vremenskom trenutku trajektorije. Funkcija *pin.exp3* služi da rotacioni vektor pretvori u rotacionu matricu.

Formiranje greške pozicije i orijentacije

Kako bi robot mogao da se približi željenoj pozi potrebno je prvo formirati grešku između željene i trenutne poze.

Greška pozicije definiše se kao:

$$e_X = x_d - x_{act}$$

gde je:

- x_d - željena pozicija
- x_{act} - stvarna pozicija

Greška orijentacije je komplikovanija i računa se pomoću logaritamske mape:

$$e_R = pin.log3(R_{act}^T * R_d)$$

gde je:

- R_d - željena orijentacija
- R_{act} - stvarna orijentacija

Ovakav način računanja greške veoma je važan jer matrice rotacije ne predstavljaju linearne veličine i nije moguće prosto oduzimati rotacione matrice. Greška orijentacije predstavlja minimalnu rotaciju potrebnu da se trenutna orijentacija poravna sa željenom.

Kompletna greška sistema formira se spajanjem pozicione i orijentacione greške:

$$e = \begin{bmatrix} e_X \\ e_R \end{bmatrix}$$

4) Inverzna kinematika

Nakon što je u operacionom prostoru generisana željena trajektorija pozicije i orijentacije TCP-a, potrebno je preći u prostor zglobova kako bi bilo moguće upravljati robotom pomoću momenata u zglobovima. Ovaj proces naziva se inverzna kinematika. Cilj inverzne kinematike jeste da se za zadatu željenu pozu TCP-a odredi konfiguracija zglobova takva da robot postigne željenu poziciju i orijentaciju.

Osnovu inverzne kinematike predstavlja Jakobijan robota. Jakobijan predstavlja vezu između brzina zglobova i brzina TCP-a. Data veza se zapisuje kao:

$$v = J(q) * dq$$

Prve tri vrste Jakobijana odnose se na translaciju TCP-a, dok se poslednje tri vrste odnose na rotaciju TCP-a. U projektu je Jakobijan računat korišćenjem Pinocchio biblioteke pomoću funkcije *pin.computeFrameJacobian().*

Pošto je u projektu korišćen Panda robot koji ima 7 stepeni slobode, u tom slučaju Jakobijan nije kvadratna matrica i ne postoji njegova inverzija. Zbog toga je korišćen pseudo-inverzni Jakobijan konkretno damped least squares (DLS) metoda pošto je takav Jakobijan stabilniji blizu singulariteta.

$$J^{\dagger} = J^{T}(J J^{T} + \lambda^{2} I)^{-1}$$

Damping faktor λ je veoma važan jer sprečava numeričku nestabilnost u blizini singularnih konfiguracija robota. Bez damping člana matrica $J J^{T}$ može postati skoro singularna, što dovodi do veoma velikih brzina zglobova i nestabilnosti sistema.

Nakon formiranja greške i pseudo-inverzije Jakobijana računa se promena uglova zglobova:

$$\Delta q = \alpha J^{\dagger} e$$

gde je α faktor konvergencije.

Nova konfiguracija robota se dobija kao:

$$q_{k+1} = q_{k} + \Delta q$$

U Pinocchio biblioteci koristi se funkcija *pin.integrate()* koja omogućava pravilnu integraciju konfiguracije robota.

Nakon rešavanja inverzne kinematike u svakom vremenskom koraku dobija se nova referentna konfiguracija zglobova q_{d} . Pošto inverse dynamics kontroler zahteva i referentne brzine i ubrzanja zglobova, one se računaju numeričkom diferencijacijom. Referentna brzina ne računa se prostim oduzimanjem konfiguracija, već pomoću funkcije *pin.difference*, koja računa razliku između dve konfiguracije u tangentnom prostoru modela robota. Nakon toga, referentno ubrzanje se dobija kao numerički izvod referentne brzine. Na ovaj način se iz sekvence referentnih konfiguracija dobijaju q_{d}, dq_{d} i ddq_{d}, koje zatim ulaze u inverse dynamics kontroler.

5) Dinamika - Inverse Dynamics Control

Nakon što su pomoću inverzne kinematike određene referentne vrednosti u prostoru zglobova, sledeći korak jeste upravljanje robotom pomoću dinamike. U ovom projektu korišćen je Inverse Dynamics Control (IDC). IDC predstavlja model-based kontroler koji koristi kompletan dinamički model robota.

Za razliku od običnog PD regulatora, IDC:

- kompenzuje gravitaciju,
- kompenzuje centrifugalne sile,
- kompenzuje Koriolisove sile,
- uzima u obzir međusobni uticaj zglobova.

zbog toga omogućava mnogo preciznije praćenje trajektorije.

Dinamika robota može se zapisati kao:

$$M(q)ddq + h(q, dq) = \tau$$

gde je:

- $M(q)$ - matrica inercije robota
- $h(q, dq)$ - član koji predstavlja nelinearne efekte sistema (gravitacija, centrifugalne sile, Koriolisove sile)
- τ - vektor koji predstavlja momente koji se zadaju motorima.

U projektu je matrica inercije računata pomoću CRBA algoritma *pin.crba()* dok su nelinearni efekti računati pomoću *pin.nonLinearEffects().* Na ovaj način u svakom simulacionom koraku dobijaju se $M(q)$ i $h(q, dq)$ za trenutno stanje robota.

Pošto IDC radi u prostoru zglobova potrebno je prvo formirati greške. Greška pozicije zglobova se računa kao:

$$e_q = q_d - q$$

gde je:

- q_d - referentna (željena) konfiguracija
- q - trenutna konfiguracija

Greška brzine:

$$e_{dq} = dq_d - dq$$

gde je:

- dq_d - referentna (željena) brzina
- dq - trenutna brzina

Na osnovu grešaka pozicije i brzine formira se željeno ubrzanje:

$$ddq_{cmd} = ddq_d + K_d e_{dq} + K_p e_q$$

gde su:

- K_p matrica proporcionalnih pojačanja,
- K_d matrica diferencijalnih pojačanja.

dok prvi član ddq_d predstavlja feedforward deo sistema. Ostatak predstavlja PD regulaciju greške.

Nakon računanja željenog ubrzanja koristi se inverse dynamics jednačina:

$$\tau = M(q)ddq_{cmd} + h(q, dq)$$

Ova jednačina predstavlja glavni kontrolni zakon projekta.

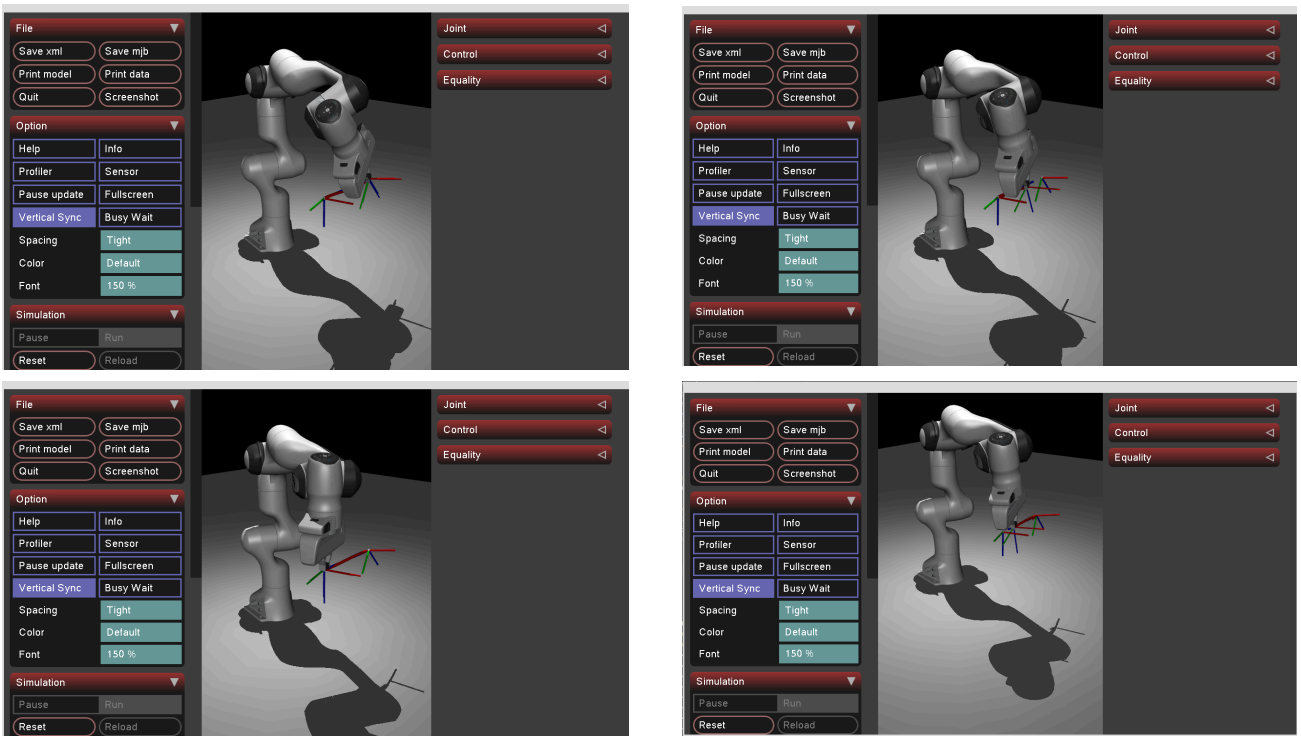
Kontroler praktično:

- računa kakvo ubrzanje želimo,
- koristi dinamički model robota,
- određuje koje momente motori moraju da proizvedu.

6) Rezultati simulacije

Na ***slici 1*** su prikazane početna i krajnja konfiguracija robotskog manipulatora Franka Emika Panda tokom izvršavanja trajektorije. Koordinatni sistemi prikazani na TCP-u predstavljaju: crvenu osu — X osu, zelenu osu — Y osu, plavu osu — Z osu.

Može se primetiti da tokom trajektorije dolazi do simultane promene pozicije i orijentacije TCP-a što je i bio zadatak ovog projekta.



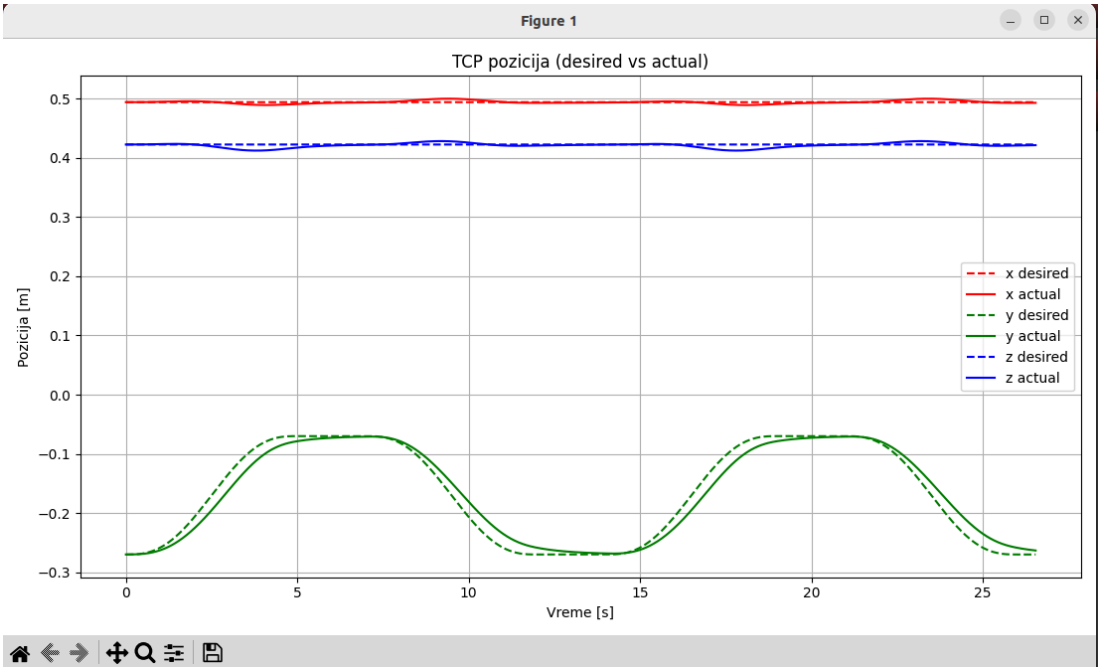
Slika 1. *Vizuelizacija početne i krajnje poze TCP-a u MuJoCo simulacionom okruženju.*

Na ***slici 2*** su prikazane željene pozicije TCP-a (desired) i ostvarene pozicije TCP-a (actual) po X, Y i Z koordinatama. Može se primetiti da stvarna trajektorija veoma dobro prati željenu trajektoriju tokom celog kretanja robota. Najveća promena ostvarena je po Y koordinati, dok su X i Z koordinate ostale skoro konstantne. To je očekivano ponašanje jer je translaciona trajektorija dominantno definisana duž Y pravca.

Mala odstupanja između desired i actual krivih predstavljaju dinamičku grešku sistema i posledica su:

- dinamike robota,
- ograničenja upravljačkog sistema,
- numeričke integracije simulacije.

I pored toga može se zaključiti da IDC kontroler omogućava veoma precizno praćenje trajektorije.

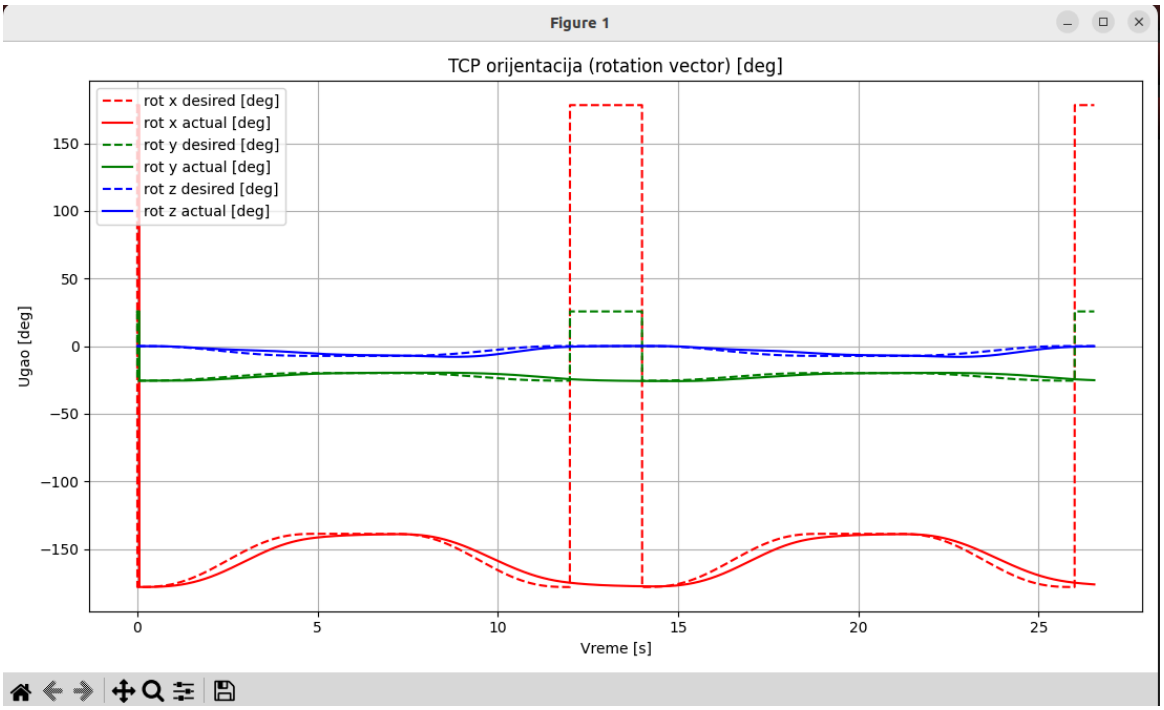


Slika 2. *Poređenje željene i ostvarene pozicije TCP-a.*

Na ***slici 3*** su prikazane komponente rotacionog vektora izražene u stepenima za željenu orijentaciju TCP-a (desired) i ostvarenu orijentaciju TCP-a (actual). Može se primetiti da robot uspešno prati željenu orijentaciju tokom trajektorije, pri čemu su odstupanja između željene i ostvarene vrednosti relativno mala. Veća odstupanja prisutna su tokom intervala ubrzavanja i usporavanja trajektorije, što je očekivano zbog:

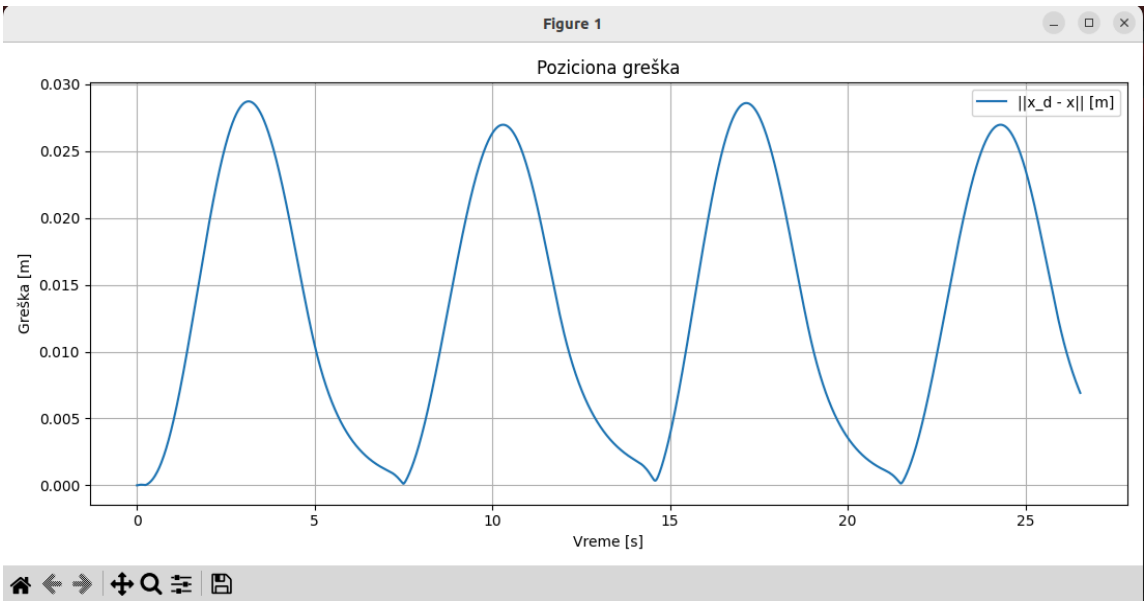
- inercije sistema,
- nelinearne dinamike robota,
- ograničenja regulatora.

Ipak, sistem ostaje stabilan tokom celog izvršavanja trajektorije.



Slika 3. *Poređenje željene i ostvarene orijentacije TCP-a.*

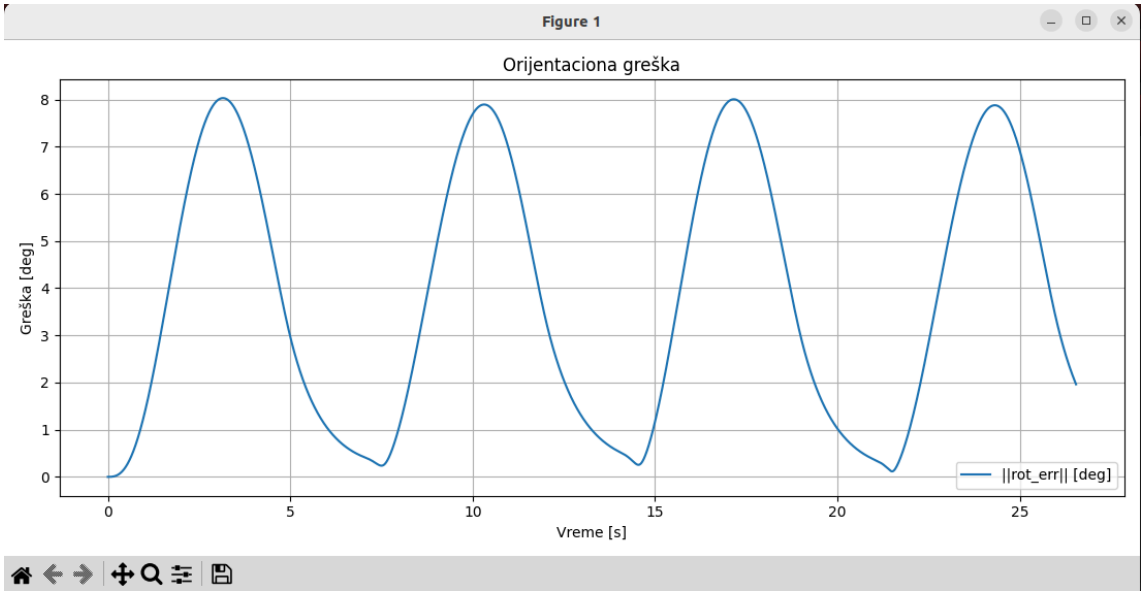
Na ***slici 4*** je prikazana norma pozicione greške. Može se primetiti da greška ostaje relativno mala tokom cele simulacije i da nema divergencije sistema. Maksimalna poziciona greška iznosi približno *0.03 m* što pokazuje da sistem ostvaruje veoma dobro praćenje translacione trajektorije. Periodični oblik greške posledica je ponavljanja trajektorije između dve target poze.



Slika 4. Norma pozicione greške TCP-a tokom simulacije.

Na ***slici 5*** je prikazana norma greške orijentacije izražena u stepenima. Može se primetiti da sistem uspešno održava stabilno praćenje željene orijentacije bez pojave

nestabilnosti ili divergencije. Maksimalna greška orijentacije iznosi približno 8° što predstavlja zadovoljavajući rezultat imajući u vidu da greška raste na tu vrednost tokom dinamičkih delova trajektorije, odnosno tokom ubrzavanja i promene pravca kretanja, dok se u stacionarnim delovima smanjuje gotovo na nulu.



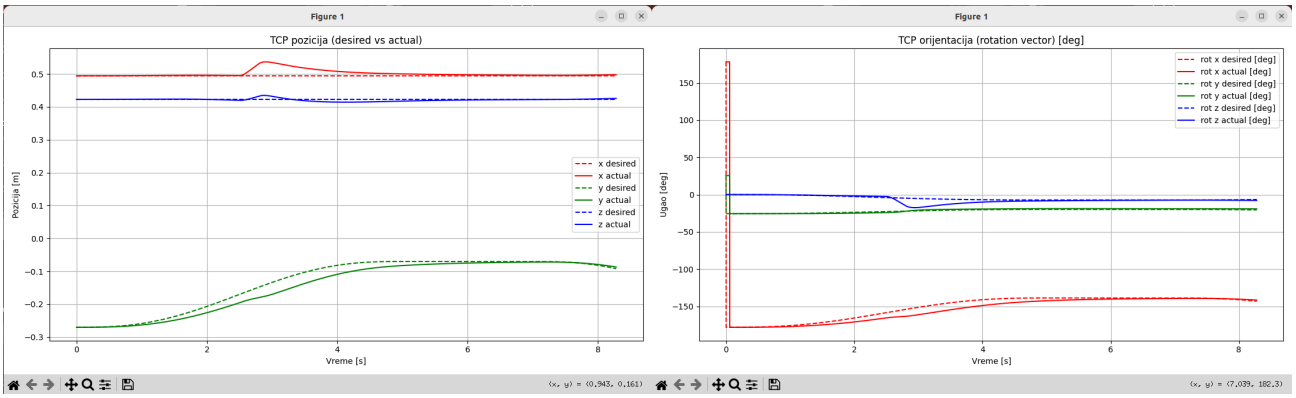
Slika 5. Norma greške orijentacije TCP-a tokom simulacije.

Test otpornosti na spoljašnje poremećaje

Tokom simulacije izvršen je dodatni test robusnosti kontrolera. U određenom trenutku simulacije (oko t=2.5 s) na TCP robota primenjena je spoljašnja sila u pozitivnom smeru X ose, čime je namerno izazvano odstupanje od željene trajektorije. Cilj ovog testa bio je da se proveri sposobnost inverse dynamics kontrolera da kompenzuje spoljašnji poremećaj i vrati sistem na zadatu trajektoriju. Rezultati simulacije pokazali su da je nakon kratkotrajnog odstupanja došlo do stabilizacije sistema, pri čemu je robot uspešno nastavio praćenje željene pozicije i orijentacije TCP-a bez pojave nestabilnosti ili divergencije.

Ukupni rezultati simulacije pokazuju da implementirani sistem uspešno realizuje:

- sintezu trajektorije u operacionom prostoru,
- inverznu kinematiku,
- inverse dynamics upravljanje,
- stabilno praćenje pozicije i orijentacije TCP-a.



Slika 6. Praćenje željene i ostvarene TCP pozicije i orijentacije tokom dejstva spoljašnje sile.

7) Zaključak

U ovom projektu realizovan je kompletan sistem za upravljanje robotskim manipulatorom Franka Emika Panda u MuJoCo simulacionom okruženju. Sinteza trajektorija pozicije i orijentacije generisana je u operacionom prostoru, dok je upravljanje izvršeno u prostoru zglobova pomoću Inverse Dynamics Control metode. Korišćenjem inverzne kinematike omogućeno je preslikavanje željene TCP poze u referentne uglove zglobova, dok je IDC kontroler omogućio dinamički konzistentno praćenje referentne trajektorije. Rezultati simulacije pokazuju da sistem stabilno prati željenu poziciju i orijentaciju TCP-a, uz male i ograničene greške tokom dinamičkih delova trajektorije.